Vanna frontend integration: basis vaults and Perena vaults

Oct 8, 2026 · @Opinder Preet Singh

Overview

Every Deposit and Withdraw button on these pages sends one Vanna instruction, `margin_execute`, which calls the vault on behalf of the user's margin account. The vault's shares land in the margin and count as collateral at the value Vanna's oracle gives them.

1. The user borrows USDC on their Vanna margin account, against any collateral Vanna accepts.
2. Deposit: the frontend sends `margin_execute` with the vault's own `deposit` instruction inside. Vanna's validator checks the call (only `deposit` / `withdraw`, the margin as the vault's user, USDC out, shares in).
3. The vault takes the USDC from the margin. A basis vault buys the asset through Jupiter and opens an equal short on Phoenix; a Perena vault mints USD* or USD* Junior. The shares go to the margin's own token account for that share mint.
4. Vanna checks that no more USDC left than the amount, that the shares arrived, and that the health factor is still above 1.10.
5. Withdraw is the same instruction with the vault's `withdraw` inside: shares leave the margin, USDC comes back to the margin's USDC account.

Who does what:

Program	Address	Role on these pages
Vanna credit layer	`BZ812nUv4Qhr2p1JVgmoJGjYTGk1brAXckyhFSCNH3Zg`	Margin account, borrowing, `margin_execute`, health factor, liquidation
Vanna oracle	`FXY5DRfekMUTbUp4uyCFpCZmCccrp6kPq3hTM4GRihnc`	Values every share as collateral (`get_price`)

Program	Address	Role on these pages
Vanna validator	`6fND3vhtstp486iE6rsSNVUox3kcjNNRsSLAN7ZPs7th`	Allows only the vaults' deposit and withdraw calls
Vanna basis vaults	`GtuQoYxzytGKtLqQGkYrVbsaPHTZN5jboP25DrSmKvGr`	The five delta-neutral vaults (SOL, ETH, BTC, TSLA, META)
Perena Vault V2	`save8RQVPMWNTzU18t3GBvBkN9hT7jsGjiCQ28FpD9H`	USD* and USD* Junior

The basis vault program is deployed on the Surfpool mainnet fork only, not on mainnet yet. Perena is the live mainnet program. Everything below is read and written from the browser with `@solana/web3.js`; nothing here needs a new Rust function.

Addresses and constants

Each vault has its own share token; a margin holds each one in its own associated token account (SPL Token, owner = the margin PDA). The receipt names (vbSOL and so on) are display names only: the share mints carry no token metadata.

Basis vaults (Surfpool fork; every address is the program's own PDA: vault = `["basis_vault", asset mint]`, share mint = `["share_mint", vault]`)

Vault	Receipt (display)	Basis vault	Share mint	Asset held	Asset mint	Asset decimals
SOL	vbSOL	`BLeEcGjJnbGQyDBeTgckreKnKD87xFDDBFTfQYEyZ1j7`	`zMv5AVEURXUVBNmTqvV7ecgXun4uRAq8m3JM13iW71B`	native SOL inside its Phoenix trader	`So11111111111111111111111111111111111111112`	9
ETH	vbETH	`7PwzomohGgy1WQeDRaJQs31UQqhapJk67gCg4zoL7znv`	`6Xin8G4Lq9RPxEDc1NnZ4JAkqwLstnosbnu3AeQavVuZ`	Wormhole ETH	`7vfCXTUXx5WJV5JADk17DUJ4ksgau7utNKj4b963voxs`	8
BTC	vbBTC	`6qFcw8qCq2oxh1iqpYAKLw3Z7tUMj6APP4tNuHyosveZ`	`7LgRZNY3XTLx95LL1YYz4q7cFLAwupwtEjYepBPvSJ1R`	cbBTC	`cbbtcf3aa214zXHbiAZQwf4122FBYbraNdFqgw4iMij`	8
TSLA	vbTSLA	`J23xTNSTgiWdVMgcDbQgJMRc5MWhQ4Q2TrcnbDwG8ejw`	`AzYyMurfA6r5hhBT2R294Zx6pMdeZfCYJ48AkcEnc43T`	TSLAx (Token-2022)	`XsDoVfqeBukxuZHWhdvWHBhgEHjGNst4MLodqsJHzoB`	8
META	vbMETA	`3P5p9a4pr6NqRtZpbpPMhC2Kgg5AFjCHishuxP6FW1pJ`	`3EfoqbFDf7FSgtifJsyFiG1QFFStf72tm2n8iqyBh5pa`	METAx (Token-2022)	`Xsa62P5mvPszXL1krVUnU5ar38bBSVcWAB6fmPCo5Zu`	8

Shares have 6 decimals. Basis program config PDA (`["config"]`): `FiyCkH6jbMkoq6ybhJWTxDKVoUULNgwcSWjZfz1M81y4`.

Perena vaults (mainnet)

Vault	Share mint	Priced from	Decimals
USD*	`star9agSpjiFe3M49B3RniVU4CMBBEK3Qnaqn3RGiFM`	Vault `ECJGrTZ6QYMEwiEAnL4oReWF126uc22e9Lojy9qyCjHT`	6
USD* Junior	`3AJYCjrhKitmBd8tRUurPA5dGHCAQh6A8L9DkuSC37Rm`	Tranche state `FbsiAonbNrC3EKWMrZxx9iSc5Q7e7zSAkro4tD8dQQJS`	6

Other Perena accounts the calls need: vault oracle `2YH9QB7qmEBgekeXH4C36L2dwMPfLdoPi5DDxeTi1kyP`, fee vault `EAf7PeuCxhMN9gEhE1Gagmj28krmu3wEqCLMbqpSTVqg`.

Shared

Name	Address
USDC	`EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v` (6 decimals)
Phoenix	`EtrnLzgbS7nMMy5fbD42kXiUzGg8XQzJ972Xtk1cjWih`
Phoenix global config	`2zskx2iyCvb6Stg7RBZkt1f6MrF4dpYtMG3yMvKwqtUZ`
Phoenix Hawkeye (views)	`RiSeVw3ZjNfsaXPRb4mgaqYaEEt41pNNJoDvVh7pgQj`
Ember (USDC gateway)	`EMBERpYNE6ehWmXymZZS2skiFmCa9V5dp14e1iduM5qy`
Jupiter v6	`JUP6LkbZbjS1jKKwapdHNy74zcZ3tLUZoi5QNyVTaV4`
Vanna market book (Phoenix)	PDA `["market_book", Phoenix global config]` of the Vanna oracle
Vanna price book	PDA `["price_book"]` of the Vanna oracle

All of these are exported from `scripts/src/basis.ts` (`BASIS_SHARES`, `BASIS`, `HAWKEYE`) and `scripts/src/perena.ts` (`PERENA_SHARES`, `PERENA`).

Setup

The frontend uses two TypeScript packages: the basis vault's own client for vault data, and Vanna's helpers for the margin side.

Package	Where	Used for
`@vanna/phoenix-vault-client`	`basis-trading-solana-phoenix/client`	Vault state, NAV, share price, APY, funding, positions, caps, events (`getVaultPage`, `getVaultState`, `getUserSummary`, `jupiterRoute`)
Vanna helpers	`vanna-credit-layer/scripts/src/basis.ts`, `perena.ts`, `devnet-positions.ts`	Share value as Vanna counts it, the `margin_execute` call, health accounts, Perena calls
Vanna IDLs	`vanna-credit-layer/target/idl/vanna_credit_layer.json`, `vanna_oracle.json`	Anchor `Program` objects for the credit layer and the oracle

Dependencies: `@solana/web3.js` 1.x, `@solana/spl-token` 0.4, `@coral-xyz/anchor` (the Vanna IDLs) and `@anchor-lang/core` 1.2 (the vault client).

```
import { Connection, PublicKey } from "@solana/web3.js";
import { AnchorProvider, Program } from "@coral-xyz/anchor";
import { vaultProgram, fetchVault, fetchPhoenixKeys, ASSETS } from
"@vanna/phoenix-vault-client";
import vannaIdl from "./idl/vanna_credit_layer.json";

export const RPC = "http://127.0.0.1:8899";            // Surfpool fork
today; a paid mainnet RPC later
export const connection = new Connection(RPC, "confirmed");

// Any funded system account: the fee payer of read-only simulations when no
wallet is connected.
export const VIEW_PAYER = new PublicKey("<any funded address>");

export const basisProgram = vaultProgram(connection);                  //
basis vault program, read-only
```

```
export const vanna = new Program(vannaIdl as any, new
AnchorProvider(connection, wallet, {}));

export const phoenix = await fetchPhoenixKeys();                          //
Phoenix's index accounts; cache a few minutes
export const basisVaults = {
  SOL:  await fetchVault(basisProgram, ASSETS.SOL.mint),
  ETH:  await fetchVault(basisProgram, ASSETS.ETH.mint),
  BTC:  await fetchVault(basisProgram, ASSETS.BTC.mint),
  TSLA: await fetchVault(basisProgram, ASSETS.TSLA.mint),
  META: await fetchVault(basisProgram, ASSETS.META.mint),
};
```

- `fetchVault` returns every address a vault's instructions need (`VaultInfo`: authority, share mint, token accounts, Phoenix trader, market, caps). Nothing has to be derived by hand.
- `fetchPhoenixKeys()` reads Phoenix's trader index accounts from `https://perp-api.phoenix.trade`. They grow as Phoenix grows; never hard-code them.
- Funding history and open interest come from Phoenix's API, which serves mainnet data even when the page runs on the fork.
- Every write is a v0 transaction with address lookup tables: one table per vault (made once by ops with `createVaultLookupTable`), the Jupiter route's own tables, and one for Vanna's accounts (protocol config, asset configs, oracle accounts).

Basis vault page: every field and where it comes from

One call, `getVaultPage(connection, basisProgram, info, payer, phoenix)`, reads almost everything on the page; the rest comes from share-price history, the margin account and Vanna's oracle. `page` below is its result, `info` the vault's `VaultInfo`. USD amounts are `bigint` with 6 decimals, rates are percents as `number`. Functions not in the vault client are written out in the next section.

Area	UI field	Function	Source and formula
Header	Vault APY (7D)	`apyFromHistory(prices, 7)`, else `page.apy7d`, else `page.yield7d.netPct`	Share-price growth over 7 days, compounded. `page.apy7d` uses the share price at the newest deposit at least 7 days old; the last fallback is an estimate from funding
Header	Vault APY (30D)	`apyFromHistory(prices, 30)`, else `page.apy30d`	Same over 30 days

Area	UI field	Function	Source and formula
Header	Funding Now	`page.funding.nowApr`	Phoenix's last hourly rate × 8,760 for the market (SOL, ETH, BTC, TSLA, META)
Header	Net Delta	`page.positions.netDeltaPct`	(asset held − short) / short, in the market's units
Header	TVL / Cap	`page.state.nav` / `page.capacity.capUsd`	NAV = asset × Phoenix mark + Phoenix equity. Cap = the first of the vault's caps to bind (`page.capacity.binding`); the SOL vault's is 450 SOL, not a dollar amount
Statistics	Net APY 30D, 90D	`apyFromHistory(prices, 30)` , `apyFromHistory(prices, 90)`	The share price is already net of every vault cost; the program charges no fee
Statistics	Since launch (annualised, from date)	`sinceLaunch(prices)`	First share price ($1.00 at the first deposit) to now, compounded; the date is the first `deposited` event
Statistics	ROI (since launch, net)	`sinceLaunch(prices).roiPct`	share price / 1.00 − 1
Statistics	Sharpe ratio	`sharpe(prices)`	Mean daily return / standard deviation × √365
Statistics	Max drawdown	`maxDrawdown(prices)`	Largest fall from a running high of the share price
Statistics	Positive funding (days in the last 30)	`page.funding.days − page.funding.negativeDays` of `page.funding.days`	Phoenix hourly funding summed per UTC day
Statistics	Avg funding 30D	`page.funding.avg30dApr`	Mean hourly rate over 30 days × 8,760
Statistics	Vault fee	constant `0`	The program has no fee: show "None". See the corrections section
Charts	Share price (1W, 1M, 3M, All)	`sharePriceSeries(prices, range)`	`state.sharePrice` sampled hourly by a backend; on-chain holds only the current value
Charts	Funding rate	`fetchFundingHistory(symbol, days)`	Phoenix `GET /v1/funding/{symbol}/rates` , hourly points
Monthly returns	Net, ROI	`monthlyReturns(prices).roiPct`	Share price at month end / month start − 1; the current month is month-to-date
Monthly returns	Net, APY	`monthlyReturns(prices).apyPct`	The month's growth, compounded over a year
Monthly returns	Gross	`monthlyReturns(prices, funding)` with `gross`	With no vault fee, gross = net + the keeper's rebalance costs. Drop the toggle if that difference is not worth showing

Area	UI field	Function	Source and formula
Where the capital sits	Venues and shares	`capitalSplit(info, page)`	Spot vaults: the asset in the vault's own token account (`positions.long.usd`, about 50%) and USDC margin with PnL on Phoenix (`state.phoenixEquity`, about 50%). SOL vault: the SOL is Phoenix collateral (`positions.long.heldIn === "phoenixCollateral"`), about 100% on Phoenix. Jupiter holds nothing
Where the yield comes from (7 days, annualised)	Funding	`page.yield7d.fundingPct`	7-day average funding × `positions.fundingExposure` (short notional per dollar of NAV: about 1.0 for SOL, 0.5 for the others)
Where the yield comes from	Rebalancing and Phoenix fees	`page.yield7d.rebalancingPct`	What the keeper's rebalances cost, from `rebalanced` events (NAV before − after). One number: the events do not split Phoenix fees from swap costs
Where the yield comes from	Vault fee	`page.yield7d.vaultFeePct`	Always 0
Where the yield comes from	Vault APY	`page.yield7d.netPct`	The sum; an estimate, next to the realized 7D APY
Your position	Shares	`marginShares(connection, margin, info)`	The margin's associated token account for the share mint. Not the wallet
Your position	Value (counts in health)	`shareValueForVanna(connection, info, shares, payer)`	Vanna's oracle `get_price`: the value the health factor uses
Your position	Value (if withdrawn now)	`shareValue(shares, page.state.nav, page.state.supply)`	The vault's own price, before exit costs
Your position	Earnings	`marginVaultFlows(connection, margin, info)`	Value now + USDC returned − USDC spent, from Vanna's `MarginExecuted` events for this vault
Your position	Receipt	constant	vbSOL, vbETH, vbBTC, vbTSLA, vbMETA
Deposit panel	Balance and 10 / 25 / 50 / 100%	`marginUsdc(connection, margin)`	The margin's USDC, not the wallet's: the deposit spends the margin's USDC
Deposit panel	You receive (shares)	`estimateDepositShares(page.state, usdc)` while typing; `previewMarginDeposit(...)` on confirm	Estimate: value less about 10 bps of entry cost. Exact: a simulation of the real transaction
Deposit panel	Health factor after	`previewMarginDeposit(...).healthFactor`	From the simulated `MarginExecuted` event
Deposit panel	Net APY and earnings with borrowed USDC	`estimateEarnings(amount, apyPct, days, borrowAprPct)`	Vault APY − Vanna's USDC borrow rate (`borrowAprPct` from the USDC reserve)

Area	UI field	Function	Source and formula
Withdraw panel	You receive (USDC)	`previewMarginWithdraw(...).usdcOut`	Simulation: the margin's USDC after − before
Withdraw panel	Burns	`previewMarginWithdraw(...).sharesBurned`	Simulation: the margin's shares before − after. The SOL vault can burn fewer than asked (see below)
Withdraw panel	Position	`shareValue(shares, nav, supply)`	As above
Pools list	TVL, APY per vault	`getVaultState` per vault + the stored history	Five calls in parallel; cache 30 to 60 seconds
How it works	Steps	static text	1 borrow on the margin, 2 deposit (`margin_execute`), 3 shares count as collateral, 4 withdraw back to USDC and repay

Basis vault view functions

These are frontend functions, not program instructions: they read accounts, simulate transactions and do arithmetic. History-based numbers (APY windows, Sharpe, drawdown, monthly returns, the chart) need the share price over time, which the chain does not keep, so a small backend job stores it.

Share-price history (backend job + math)

Every hour, for each vault, store `{ vault, time, sharePrice, nav, supply }` from `getVaultState(connection, info, VIEW_PAYER, phoenix)`. Until enough history exists, `page.apy7d` / `page.apy30d` (from deposit events) are the fallback.

```
import { apyBetween } from "@vanna/phoenix-vault-client";

export type PricePoint = { time: number; price: bigint };   // unix seconds;
USD per whole share, 6 dp
const DAY = 86_400;
const usd = (v: bigint) => Number(v) / 1e6;

const priceAt = (prices: PricePoint[], time: number) =>
[...prices].reverse().find((p) => p.time <= time);

// Vault APY (7D), (30D), Net APY 30D / 90D
export function apyFromHistory(prices: PricePoint[], days: number, now =
Date.now() / 1000): number | null {
  const last = prices.at(-1), from = priceAt(prices, now - days * DAY);
```

```
  if (!last || !from || from.time === last.time) return null;
  return apyBetween(from.price, last.price, last.time - from.time);
}

// Since launch (annualised) and ROI: the first point is the first deposit,
at $1.00
export function sinceLaunch(prices: PricePoint[]) {
  const first = prices[0], last = prices.at(-1)!;
  return {
    launchedAt: first.time,
    roiPct: (usd(last.price) / usd(first.price) - 1) * 100,
    apyPct: apyBetween(first.price, last.price, last.time - first.time),
  };
}

// The last sample of each UTC day
const daily = (prices: PricePoint[]) => [...new Map(prices.map((p) =>
[Math.floor(p.time / DAY), p])).values()];

// Sharpe ratio: daily returns, annualised, no risk-free rate
export function sharpe(prices: PricePoint[]): number | null {
  const d = daily(prices);
  const r = d.slice(1).map((p, i) => usd(p.price) / usd(d[i].price) - 1);
  if (r.length < 2) return null;
  const mean = r.reduce((a, b) => a + b, 0) / r.length;
  const sd = Math.sqrt(r.reduce((a, b) => a + (b - mean) ** 2, 0) / (r.length
- 1));
  return sd === 0 ? null : (mean / sd) * Math.sqrt(365);
}

// Max drawdown, in percent (negative)
export function maxDrawdown(prices: PricePoint[]): number {
  let peak = 0, worst = 0;
  for (const p of prices) {
    peak = Math.max(peak, usd(p.price));
    worst = Math.min(worst, usd(p.price) / peak - 1);
  }
  return worst * 100;
}

// Monthly returns: each month opens at the previous month's last price
export function monthlyReturns(prices: PricePoint[]) {
  const months = new Map<string, { open: PricePoint; close: PricePoint }>();
```

```
  let prev: PricePoint | undefined;
  for (const p of prices) {
    const key = new Date(p.time * 1000).toISOString().slice(0, 7);   //
"2026-09"
    const m = months.get(key);
    if (m) m.close = p; else months.set(key, { open: prev ?? p, close: p });
    prev = p;
  }
  return [...months].map(([month, { open, close }]) => ({
    month,
    roiPct: (usd(close.price) / usd(open.price) - 1) * 100,
    apyPct: apyBetween(open.price, close.price, close.time - open.time),
  }));
}

// Share price chart
const RANGE_DAYS = { "1W": 7, "1M": 30, "3M": 90, All: Infinity } as const;
export const sharePriceSeries = (prices: PricePoint[], range: keyof typeof
RANGE_DAYS, now = Date.now() / 1000) =>
  prices.filter((p) => p.time >= now - RANGE_DAYS[range] * DAY).map((p) => ({
time: p.time, price: usd(p.price) }));
```

Where the capital sits

```
import type { VaultInfo, VaultPage } from "@vanna/phoenix-vault-client";

export function capitalSplit(info: VaultInfo, page: VaultPage) {
  const { nav, assetValue, phoenixEquity } = page.state;
  const pct = (v: bigint) => (nav > 0n ? (Number(v) / Number(nav)) * 100 :
0);
  if (info.kind === "nativeSol") {
    return [{ venue: "Phoenix", holds: "SOL collateral, USDC and PnL", usd:
nav, pct: 100 }];
  }
  return [
    { venue: "Vault token account", holds: "Spot asset", usd: assetValue,
pct: pct(assetValue) },
    { venue: "Phoenix", holds: "USDC margin and PnL", usd: phoenixEquity,
pct: pct(phoenixEquity) },
  ];
}
// The short itself: page.positions.short.usd (notional) and
page.positions.short.marketUnits.
```

The margin's position, its value and its earnings

```
import { Connection, PublicKey } from "@solana/web3.js";
import { getAssociatedTokenAddressSync } from "@solana/spl-token";
import { Program, utils } from "@coral-xyz/anchor";
import { BASIS, BASIS_SHARES, basisShareValue } from "./vanna/basis";
// vanna-credit-layer/scripts/src/basis.ts

const VANNA = new PublicKey("BZ812nUv4Qhr2p1JVgmoJGjYTGk1brAXckyhFSCNH3Zg");
const USDC = new PublicKey("EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v");
export const marginOf = (wallet: PublicKey) =>
PublicKey.findProgramAddressSync([Buffer.from("margin"), wallet.toBuffer()],
VANNA)[0];
const ataOf = (owner: PublicKey, mint: PublicKey) =>
getAssociatedTokenAddressSync(mint, owner, true);

async function balance(connection: Connection, account: PublicKey):
Promise<bigint> {
  const b = await connection.getTokenAccountBalance(account).catch(() =>
null);
```

```
  return b ? BigInt(b.value.amount) : 0n;
}
export const marginShares = (c: Connection, margin: PublicKey, shareMint:
PublicKey) => balance(c, ataOf(margin, shareMint));
export const marginUsdc = (c: Connection, margin: PublicKey) => balance(c,
ataOf(margin, USDC));

// What the shares count for in the health factor: Vanna's oracle, simulated
(signs nothing).
// basisShareValue returns USD with 9 decimals; divide by 1,000 for 6.
export async function shareValueForVanna(c: Connection, key: keyof typeof
BASIS_SHARES, shares: bigint, payer: PublicKey) {
  return (await basisShareValue(c, BASIS_SHARES[key], shares, payer)) /
1_000n;
}

// Earnings: USDC the margin put into this vault and got back, from Vanna's
MarginExecuted events.
// They are emitted with emit_cpi!, so they sit in inner instructions and
survive log truncation.
const EVENT_IX_TAG = Buffer.from([0xe4, 0x45, 0xa5, 0x2e, 0x51, 0xcb, 0x9a,
0x1d]);

export async function marginVaultFlows(c: Connection, vanna: Program, margin:
PublicKey, shareMint: PublicKey) {
  const signatures = (await c.getSignaturesForAddress(ataOf(margin,
shareMint), { limit: 200 }))
    .filter((s) => !s.err).map((s) => s.signature);
  let spentUsdc = 0n, receivedUsdc = 0n;
  for (const tx of await c.getTransactions(signatures, {
maxSupportedTransactionVersion: 0 })) {
    if (!tx?.meta) continue;
    const keys = tx.transaction.message.getAccountKeys({
accountKeysFromLookups: tx.meta.loadedAddresses });
    for (const ix of (tx.meta.innerInstructions ?? []).flatMap((i) =>
i.instructions)) {
      if (!keys.get(ix.programIdIndex)?.equals(VANNA)) continue;
      const data = Buffer.from(utils.bytes.bs58.decode(ix.data));
      if (!data.subarray(0, 8).equals(EVENT_IX_TAG)) continue;
      const event =
vanna.coder.events.decode(data.subarray(8).toString("base64"));
      if (event?.name !== "MarginExecuted" ||
!event.data.programId.equals(BASIS)) continue;
```

```
      for (const t of event.data.spent) if (t.mint.equals(USDC)) spentUsdc +=
BigInt(t.amount.toString());
      for (const t of event.data.received) if (t.mint.equals(USDC))
receivedUsdc += BigInt(t.amount.toString());
    }
  }
  return { spentUsdc, receivedUsdc };
}
// earnings = value now + receivedUsdc − spentUsdc. Shares that left by
liquidation or arrived by
// transfer are not in these flows: flag the number as approximate when that
happened.
```

Vanna's USDC borrow rate (for "Net APY with borrowed USDC")

```
import { borrowRatePerSecondWad, utilizationWad, SECONDS_PER_YEAR, WAD } from
"./vanna/devnet-math";

export async function usdcBorrowAprPct(vanna: Program): Promise<number> {
  const [reserve] = PublicKey.findProgramAddressSync([Buffer.from("reserve"),
USDC.toBuffer()], VANNA);
  const r: any = await (vanna.account as any).reserve.fetch(reserve);
  const big = (v: any) => BigInt(v.toString());
  const util = utilizationWad(big(r.accountedLiquidityAssets),
big(r.totalBorrowAssets));
  const curve = { linearCoeffWad: big(r.rateCurve.linearCoeffWad),
jumpCoeffWad: big(r.rateCurve.jumpCoeffWad), rateMultiplierWad:
big(r.rateCurve.rateMultiplierWad) };
  return Number((borrowRatePerSecondWad(curve, util) * SECONDS_PER_YEAR *
10_000n) / WAD) / 100;
}
// estimateEarnings(amount, vaultApyPct, days, await usdcBorrowAprPct(vanna))
gives the panel's net APY.
```

The deposit and withdraw previews (`previewMarginDeposit` , `previewMarginWithdraw`) simulate the real transaction; they are in the next section.

Basis deposit and withdraw through the margin

The user signs one v0 transaction whose main instruction is Vanna's `margin_execute(data, call_account_count)`. `data` is the vault's own `deposit` or `withdraw` instruction; the vault never sees the wallet, only the margin.

Arguments

Field of `data`	Type	Deposit	Withdraw
discriminator	8 bytes	`[242,35,198,137,82,225,242,182]`	`[183,18,70,156,148,109,161,34]`
`gti`, `atb`	u8, u8	Number of Phoenix global trader index and active trader buffer accounts	same
amount	u64	`usdc_amount` (6 dp)	`shares` (6 dp)
minimum	u64	`min_shares`: fail below this	`min_usdc_out`: fail below this
route	struct / option	`{ accounts: u8, data: bytes }`: a Jupiter route built for the vault authority	`0` (no swap) or `1` + route (to sell the leaver's part of the asset)

`call_account_count` = 32 + `gti` + `atb` + `route.accounts`. Accounts, in order: Vanna's named accounts (wallet as `authority`, protocol config, margin, the integration PDA for the basis program, the basis program, the validator, no venue, then the event authority and Vanna program that `emit_cpi!` needs), then the vault's 32 accounts with the margin as `user` (index 0), the margin's share account at 25 and its USDC account at 26, then Phoenix's index accounts, the route's accounts, the margin's health groups and the oracle's accounts. `basisCallData` and `basisCallAccounts` in `scripts/src/basis.ts` build `data` and the vault's part exactly.

Building the instructions

```
import { ComputeBudgetProgram, PublicKey, SystemProgram } from
"@solana/web3.js";
import { NATIVE_MINT, createAssociatedTokenAccountIdempotentInstruction }
from "@solana/spl-token";
import { BASIS, BASIS_SHARES, BasisKey, basisCallAccounts, basisCallData,
basisRoute,
         depositSwapAmount, readBasisConfig, readBasisVault, withdrawEstimate
} from "./vanna/basis";
```

```
import { readExchange } from "./vanna/phoenix";
import { buildRemainingAccounts } from "./vanna/devnet-positions";

const VALIDATOR = new
PublicKey("6fND3vhtstp486iE6rsSNVUox3kcjNNRsSLAN7ZPs7th");
const pda = (seeds: Buffer[]) => PublicKey.findProgramAddressSync(seeds,
VANNA)[0];

export async function marginBasisInstructions(
  vanna: Program, wallet: PublicKey, key: BasisKey, call: "deposit" |
"withdraw",
  amount: bigint, minOut: bigint, slippageBps = 50,
) {
  const c = vanna.provider.connection;
  const share = BASIS_SHARES[key];
  const margin = marginOf(wallet);
  const [vault, config, exchange] = await Promise.all([readBasisVault(c,
share.vault), readBasisConfig(c), readExchange(c)]);
  if (call === "deposit" && vault.native) throw new Error("The SOL vault
takes deposits from a wallet only (see below)");

  const sold = call === "deposit" ? depositSwapAmount(amount,
vault.targetMarginBps) : await withdrawEstimate(c, vault, amount);
  const route = sold > 0n ? await basisRoute(vault, config, call ===
"deposit", sold, slippageBps) : null;
  const cpi = basisCallAccounts(vault, config, exchange, margin, route);
  const rest = await buildRemainingAccounts(vanna, margin, { priced: ["usdc",
key] });

  const execute = await vanna.methods
    .marginExecute(basisCallData(call, exchange, amount, minOut, route),
cpi.length)
    .accounts({
      authority: wallet,
      protocolConfig: pda([Buffer.from("protocol")]),
      marginAccount: margin,
      integration: pda([Buffer.from("integration"), BASIS.toBuffer()]),
      targetProgram: BASIS,
      validator: VALIDATOR,
      venueAsset: null,
    } as any)                                    // eventAuthority and program
are resolved from the IDL
    .remainingAccounts([...cpi, ...rest])
```

```
    .instruction();

  const setup = [
    ComputeBudgetProgram.setComputeUnitLimit({ units: 1_400_000 }),
    ComputeBudgetProgram.requestHeapFrame({ bytes: 256 * 1024 }),   //
without it the call runs out of heap
    createAssociatedTokenAccountIdempotentInstruction(wallet, ataOf(margin,
share.mint), margin, share.mint),
  ];
  if (vault.native) {   // SOL withdrawals: the vault opens and closes this
account, and a margin cannot pay its rent
    setup.push(createAssociatedTokenAccountIdempotentInstruction(wallet,
ataOf(vault.authority, NATIVE_MINT), vault.authority, NATIVE_MINT));
  }
  return { instructions: [...setup, execute], jupiterTables:
route?.lookupTables ?? [] };
}
```

Compile it into a v0 message with the route's lookup tables plus the vault's table and a Vanna table (protocol config, asset configs, oracle accounts). A margin deposit or withdrawal names about 60 accounts; a legacy transaction cannot hold it.

Previews (exact numbers before signing)

```
import { Connection, VersionedTransaction } from "@solana/web3.js";

export async function previewMargin(c: Connection, vanna: Program, tx:
VersionedTransaction, margin: PublicKey, shareMint: PublicKey) {
  const watched = [ataOf(margin, shareMint), ataOf(margin, USDC)];
  const before = await Promise.all(watched.map((a) => balance(c, a)));
  const sim = await c.simulateTransaction(tx, {
    sigVerify: false, replaceRecentBlockhash: true, innerInstructions: true,
    accounts: { encoding: "base64", addresses: watched.map((a) =>
a.toBase58()) },
  });
  if (sim.value.err) return { ok: false as const, err: sim.value.err, logs:
sim.value.logs ?? [] };
  const after = (sim.value.accounts ?? []).map((a) => (a ?
Buffer.from(a.data[0], "base64").readBigUInt64LE(64) : 0n));
  let healthFactorWad: bigint | null = null;
  for (const ix of (sim.value.innerInstructions ?? []).flatMap((i) =>
i.instructions) as any[]) {
```

```
    if (!ix.data || !ix.programId?.equals(VANNA)) continue;
    const data = Buffer.from(utils.bytes.bs58.decode(ix.data));
    if (!data.subarray(0, 8).equals(EVENT_IX_TAG)) continue;
    const e = vanna.coder.events.decode(data.subarray(8).toString("base64"));
    if (e?.name === "MarginExecuted") healthFactorWad =
BigInt(e.data.borrowHealthFactorWad.toString());
  }
  return {
    ok: true as const,
    shareChange: after[0] - before[0],   // deposit: shares received;
withdraw: − shares burned
    usdcChange: after[1] - before[1],    // deposit: − USDC spent; withdraw:
USDC received
    healthFactorWad,                     // 1e18 = 1.00
    unitsConsumed: sim.value.unitsConsumed,
  };
}
```

On confirm: build with `minOut = 0`, preview, then rebuild with `minOut = shares × (10,000 − slippageBps) / 10,000` (deposit) or `usdc × (…)` (withdraw) and send. Build when the user clicks, not when the page loads: a Jupiter route goes stale within seconds (`SwapAmountOff`, `SwapPriceTooLow`); rebuild and retry. If a withdrawal fails with `SwapNotNeeded` (6030), rebuild it without a route. Only the wallet signs.

Measured in the tests: an ETH-vault withdrawal through the margin used 451k compute units, a SOL-vault one 506k.

The SOL vault through a margin

- **Deposit: not possible with the current vault program.** Every SOL deposit posts native SOL to Phoenix, and Phoenix's `ReallocTrader` takes a system transfer from the vault's `user`; a margin account (a program account holding data) can never make one. The fix belongs in the vault program: pay that step from the vault authority. Until then, buy SOL-vault shares from the wallet (the vault client's `prepareDeposit`) and post them with Vanna's `user_deposit_collateral`; they count in health like the others.
- **Withdraw: works.** Create the vault authority's wrapped-SOL account in the same transaction, before the call (the builder above does). The vault refunds that account's rent (about 0.002 SOL) to the margin account.

- **Partial payouts.** When Phoenix lets only part of the SOL leave, the vault burns only the shares it paid for; the rest stay in the margin. Show `previewMargin(...).shareChange`, not the amount asked.

Health factor and risk display

Basis-vault and Perena shares count in the health factor at Vanna's oracle value, the same as any allowed token. Because a basis vault holds its asset against an equal short, its shares barely move with the asset's price: in the tests a ±25% ETH move changed a share-backed margin's health factor by under 0.4 bps.

How Vanna values a basis share. The oracle reads the vault account and its Phoenix trader, values the asset held and the short at Vanna's own index price for the market (Scope or Pyth, not Phoenix's mark), subtracts the Phoenix fee to close the short, and takes the lower of NAV / supply and the vault's own share price, (NAV + $1) / (supply + 1). The result is never above the vault's own NAV: 1.75 to 3.5 bps lower for SOL, ETH and BTC in the tests, and up to 16 bps lower for TSLA and META.

What to show	Function	Note
Health factor now	`computePositionHealth` in `scripts/src/devnet.ts` (oracle value of every holding / debt)	1e18 = 1.00. Borrowing and any margin call need it above 1.10 afterwards
Health factor after a deposit or withdrawal	`previewMargin(...).healthFactorWad`	From the simulated transaction
Value as collateral	`shareValueForVanna(...)`	What the health factor uses
Value if withdrawn	`shareValue(shares, nav, supply)`	The vault's price; the real payout is lower by the close's spread and Jupiter's price impact (about 0.2% in tests)

What to show	Function	Note
Price freshness	`PriceChecks` bits in the oracle answer (`checks & 1` = fresh)	A share's price is fresh while the USDC price and the market's index price are fresh, and no older than the share's `max_age_secs` (300 s in the scripts)

What can still lower the health factor, worth one line each in the UI:

- **Negative funding.** When shorts pay longs, the share price falls slowly. `page.funding.negativeDays` shows how often that happened.
- **Interest.** The USDC debt grows at the borrow rate (`usdcBorrowAprPct`); the net APY panel subtracts it.
- **A stale price.** A stale index price makes the shares count as zero for borrowing and margin calls (`StalePrice`), and blocks liquidation until it updates.
- **The vault's own short being liquidated on Phoenix.** That needs a large rally: `page.positions.liquidationRisePct` (about +350% for SOL, +96% for the spot vaults at 1x margin). After it, the vault holds unhedged asset until the keeper re-hedges, and the shares move with the price.
- **Wrapped-asset depeg.** Wormhole ETH and cbBTC are valued at the ETH and BTC index prices; a depeg of the wrapped token is not seen.

Liquidation. At a health factor of 1.10 or below, anyone can liquidate one debt–collateral pair per call: repay up to 50% of the debt (100% when the health factor is at or below 1 + the bonus, or the debt is worth $100 or less) and take shares worth at most the repaid value × (1 + the bonus), or × the health factor itself while the margin is solvent but at or below 1 + the bonus. The bonus is the share's `liquidation_bonus_bps` (5% in the tests; at most 5%). In the test, $1,000 repaid took exactly $1,050 of ETH-vault shares, which the liquidator redeemed at the vault for $1,047.90. Each share mint's `max_collateral_per_margin` caps how many shares count for borrowing; a liquidator can seize the whole balance. If Vanna turns a share mint's collateral switch off, its shares count as zero for borrowing, withdrawals and margin calls, but still count in liquidation.

Perena USD* and USD* Junior: page fields and view functions

Perena publishes each share's USD price on-chain, so a Perena page needs no simulation: one account read gives the price and its last update. Vanna's oracle uses that same price, so the value shown is exactly the value the health factor counts.

Prices are in USD, not USDC: Perena values its USDC holding at its own price (about $0.99985), so 1,000 USDC mints about $999.85 of shares. Do not multiply by a USDC price again.

UI field	Function	Source and formula
Share price	`perenaSharePrice(c, key).price`	USD*: vault `ECJGrTZ6…` u64 at byte 1584. Junior: tranche state `FbsiAonb…` u64 at byte 256. 1,000,000 = $1.00
NAV last updated	`perenaSharePrice(c, key).updatedAt`	USD*: i64 at byte 1592; Junior: i64 at byte 264. Vanna counts the shares only while this is younger than the share's `max_age_secs` (3,600 s in the scripts)
TVL	`perenaTvl(c, key)`	Share mint supply × share price
Vault APY (7D, 30D), ROI, Sharpe, drawdown, monthly returns, chart	the same history functions as the basis page	Store `{ time, price }` hourly from `perenaSharePrice`; the chain keeps only the current price
Exit fee	constant per vault	USD* round trip of 1,000 USDC returned between 999 and 1,000 USDC in the tests. Junior: an instant exit pays 1% (Perena's `early_unstake_fee_bps` = 100)

UI field	Function	Source and formula
Your position (shares)	`marginShares(c, margin, mint)`	The margin's token account for the share mint
Your position (value, counts in health)	`perenaValue(shares, price)`	shares × price / 1e6; identical to the oracle's value
Earnings	`marginVaultFlows(...)` with `PERENA` in place of `BASIS`	USDC in and out from Vanna's `MarginExecuted` events
Deposit: you receive	`previewMargin(...).shareChange`	Simulation; estimate while typing: usdc × 0.99985 / price
Withdraw: you receive	`previewMargin(...).usdcChange`	Simulation. Junior shows the 1% fee on this line
Health factor after	`previewMargin(...).healthFactorWad`	As for the basis vaults

Funding, net delta and the capital split have no meaning for Perena: leave those cards out of its page.

```
import { Connection, PublicKey } from "@solana/web3.js";
import { PERENA_SHARES, PerenaShareKey } from "./vanna/perena";      //
vanna-credit-layer/scripts/src/perena.ts

const PERENA_LAYOUT = {
  usdstar:  { priceAt: 1584, updatedAt: 1592 },   // Perena vault:
accounting.mint_share_price, last_update_ts
  usdstarj: { priceAt: 256,  updatedAt: 264 },    // tranche state: junior
share_price, last update
} as const;

export async function perenaSharePrice(c: Connection, key: PerenaShareKey) {
  const share = PERENA_SHARES[key];
  const info = await c.getAccountInfo(share.account);
  if (!info) throw new Error(`Perena account ${share.account.toBase58()} not
found`);
  return {
    price: info.data.readBigUInt64LE(PERENA_LAYOUT[key].priceAt),
```

```
// USD per share, 6 dp
    updatedAt:
Number(info.data.readBigInt64LE(PERENA_LAYOUT[key].updatedAt)),
  };
}

export async function perenaTvl(c: Connection, key: PerenaShareKey) {
  const [{ price }, mint] = await Promise.all([perenaSharePrice(c, key),
c.getAccountInfo(PERENA_SHARES[key].mint)]);
  const supply = mint!.data.readBigUInt64LE(36);
  return (supply * price) / 1_000_000n;
// USD, 6 dp
}

export const perenaValue = (shares: bigint, price: bigint) => (shares *
price) / 1_000_000n;

// Shown next to the value: Vanna counts the shares for borrowing only while
the NAV is fresh.
export const perenaIsFresh = (updatedAt: number, maxAgeSecs = 3_600, now =
Date.now() / 1000) =>
  Math.abs(now - updatedAt) <= maxAgeSecs;
```

Perena deposit and withdraw through the margin

The same `margin_execute`, with Perena's instant instructions inside. Perena's instructions have no minimum-out argument, so always show the simulated amount before the user signs.

Action	Perena instruction	`data`	Accounts	Spent (index)	Received (index)
USD* deposit	`execute_deposit`	`[247,103,46,184,88,188,56,46]` + USDC u64	13	margin USDC (6)	margin USD* (10)
USD* withdraw	`execute_withdraw`	`[255,93,15,141,187,94,246,162]` + shares u64	13	margin USD* (10)	margin USDC (6)
Junior deposit	`execute_tranche_deposit`	`[231,224,46,207,175,45,77,148]` + `0` (Junior) + USDC u64	15	margin USDC (6)	margin Junior (10)
Junior withdraw (instant, 1% fee)	`execute_tranche_withdraw`	`[240,82,254,246,177,29,57,126]` + `0` + shares u64	13	margin Junior (8)	margin USDC (6)

The margin is account 0 in every call. `perenaCallData` and `perenaCallAccounts` in `scripts/src/perena.ts` build both exactly.

```
import { ComputeBudgetProgram, PublicKey } from "@solana/web3.js";
import { createAssociatedTokenAccountIdempotentInstruction } from
"@solana/spl-token";
import { PERENA, PERENA_SHARES, PerenaShareKey, perenaCallAccounts,
perenaCallData } from "./vanna/perena";

export async function marginPerenaInstructions(
  vanna: Program, wallet: PublicKey, key: PerenaShareKey, call: "deposit" |
"withdraw", amount: bigint,
) {
  const share = PERENA_SHARES[key];
  const margin = marginOf(wallet);
  const cpi = perenaCallAccounts(share, call, margin);
  const rest = await buildRemainingAccounts(vanna, margin, { priced: ["usdc",
key] });
  const execute = await vanna.methods
    .marginExecute(perenaCallData(share, call, amount), cpi.length)
    .accounts({
      authority: wallet,
      protocolConfig: pda([Buffer.from("protocol")]),
      marginAccount: margin,
      integration: pda([Buffer.from("integration"), PERENA.toBuffer()]),
      targetProgram: PERENA,
      validator: VALIDATOR,
      venueAsset: null,
    } as any)
```

```
    .remainingAccounts([...cpi, ...rest])
    .instruction();
  return [
    ComputeBudgetProgram.setComputeUnitLimit({ units: 1_000_000 }),
    createAssociatedTokenAccountIdempotentInstruction(wallet, ataOf(margin,
USDC), margin, USDC),
    createAssociatedTokenAccountIdempotentInstruction(wallet, ataOf(margin,
share.mint), margin, share.mint),
    execute,
  ];
}
// Preview with previewMargin(c, vanna, tx, margin, share.mint), exactly as
for the basis vaults.
```

- A Perena deposit uses about 125k compute units; the health check adds to that.
- Vanna refuses everything else on Perena: queued (non-instant) withdrawals, the senior tranche, `_from_external` withdrawals and cranks.
- Instant liquidity is limited (about $1.35M when this was integrated). A large exit can fail on Perena's side; suggest a smaller amount.
- The Junior exit's 1% fee lowers the margin's value: an exit that would leave the health factor at 1.10 or below is refused (`HealthFactorTooLow`). Show the health factor after, from the preview.
- Pre-create the margin's share account in the same transaction: the margin cannot pay rent.

Errors

The same number means different things in different programs, so read which program failed first: the first log line `Program <id> failed: custom program error: 0x…` names the innermost program that failed.

```
export function programError(logs: string[] = []) {
  for (const line of logs) {
    const m = line.match(/^Program (\w+) failed: custom program error: (0x[0-
9a-f]+)$/);
    if (m) return { program: m[1], code: parseInt(m[2], 16) };
  }
  return null;
}
```

Program	Code	Name	Show the user
Vanna credit layer	6026	HealthFactorTooLow	"This would take your health factor below 1.10"
Vanna credit layer	6024	StalePrice	"Prices are updating; try again in a minute"
Vanna credit layer	6025, 6048	ConfidenceTooWide, PriceTooDivergentFromTwap	"The market is moving fast; try again shortly"
Vanna credit layer	6013	AssetNotCollateralEnabled	"This vault isn't open on Vanna yet" (its share mint is not on the allowed list)
Vanna credit layer	6044	IntegrationDisabled	"Deposits into this vault are paused on Vanna"
Vanna credit layer	6057	SpendLimitExceeded	"Something went wrong"; report it (the call spent more than its amount)
Vanna credit layer	6046, 6047, 6023, 6049	InvalidCallAccounts, InvalidCallResult, IncompletePositionAccounts, InvalidAgentAccounts	"Something went wrong"; a client bug: rebuild the accounts
Vanna credit layer	6004	ProtocolActionPaused	"Vanna is paused for maintenance"
Vanna credit layer	6021	TooManyAssets	"Your account holds the maximum number of positions"
Vanna validator	6000	CallNotAllowed	"This action isn't supported"
Vanna validator	6001, 6002	InvalidCallAccounts, WrongAccountCount	Client bug: rebuild the call
Vanna validator	6003	ZeroAmount	"Enter an amount"

Program	Code	Name	Show the user
Vanna oracle	6022	InvalidBasisAccount	Config bug: the share's price source names the wrong vault
Vanna oracle	6021	InvalidPerenaAccount	Config bug: the share's price source names the wrong Perena account
Vanna oracle	6020	PhoenixTraderNotSupported	"This vault can't be valued right now"; alert ops
Basis vault	6001	Paused	"Vault paused for maintenance"; withdrawals still work
Basis vault	6002	DepositsClosed	"Deposits are closed for now"
Basis vault	6012, 6013	MaxPositionReached, OpenInterestShareReached	"The vault is full"; show the room left
Basis vault	6014	DepositTooLarge	"Max {maxDepositUsd} per deposit"
Basis vault	6015	MarketAfterHours	"The TSLA / META market is closed"
Basis vault	6019	MultiplierChanging	"Deposits pause around a stock dividend or split"
Basis vault	6022, 6023, 6027, 6031, 6032	SharesTooLow, PayoutTooLow, PriceOutOfBand, SwapAmountOff, SwapPriceTooLow	"Price moved; try again" (rebuild with a fresh route)
Basis vault	6025	NothingToWithdraw	"Phoenix lets nothing leave right now; try later"

Program	Code	Name	Show the user
Basis vault	6030	SwapNotNeeded	No message: rebuild the withdrawal without a route
System program, inside a SOL-vault deposit	—	`Transfer: from must not carry data`	"Deposit into the SOL vault from your wallet" (see the SOL vault limitation)
Perena	any	—	"Perena couldn't process this; try a smaller amount" (instant liquidity)

The full basis vault list is in its `FRONTEND_INTEGRATION.md`, section 9.

Where the current design's sample data does not match the programs

Seven things on the SOL Basis page describe a product the basis program does not implement; wire them as below, or change the program first.

Design element	Sample shows	What the program does	Change in the UI
Vault fee (Statistics, yield breakdown)	10% of profit, −1.2%	No fee of any kind	Show "None" and drop the line from the yield breakdown, unless a fee is added to the program
Where the capital sits	Jupiter: spot SOL 49.8%; Phoenix the rest	Jupiter only routes the swaps and holds nothing. The SOL vault's SOL is Phoenix collateral	SOL: one row, Phoenix about 100% (SOL collateral, USDC and PnL). ETH, BTC, TSLA, META: vault token account about 50%, Phoenix about 50%

Design element	Sample shows	What the program does	Change in the UI
Funding line of the yield breakdown	26.0% on the half held short	The SOL vault's short is about the whole NAV (`fundingExposure` about 1.0); the other vaults' about half (0.5)	Label from `page.positions.fundingExposure`: "on the full short" for SOL, "on the half held short" for the others
Phoenix fees	Position and borrow fees, −0.9%	The vault borrows nothing on Phoenix. Phoenix's taker fee on keeper rebalances is in the rebalancing cost; each user's own entry and exit fees are paid in their deposit or withdrawal, not from the vault's yield	Merge "Phoenix fees" and "Execution" into one line: `page.yield7d.rebalancingPct`
Monthly returns, Gross / Net	Two series	With no fee, gross and net differ only by the keeper's rebalance costs	Keep only Net, or define Gross as net + rebalancing
Deposit on the SOL vault	Enabled after launch	A margin cannot deposit into the SOL vault with the current program (Phoenix's `ReallocTrader`)	Deposit from the wallet, then post the shares as collateral; withdraw through the margin works. ETH, BTC, TSLA and META take margin deposits
Position, balance, % buttons	Wallet-style	Deposits spend the margin's USDC; shares sit in the margin	Read the margin's USDC and share accounts (`marginUsdc`, `marginShares`), show the health factor after

Two smaller notes: the share price is USD per share at Phoenix's mark, not USDC (about the same while USDC holds its peg), and the cap is per vault and binds in different units (the SOL vault's is 450 SOL), so read `page.capacity` rather than a fixed dollar figure.

Testing checklist

Run these on a fresh Surfpool fork with the five basis share mints registered on Vanna (`register-phoenix` for each market, `register-basis`, `register-basis-share --vault <v>`) and Perena registered (`register-perena`, `register-perena-share`).

- [ ] Each basis vault page loads from `getVaultPage`; TVL, share price and net delta match `getVaultState`
- [ ] The value shown as collateral equals `shareValueForVanna` and is never above `shareValue(shares, nav, supply)`
- [ ] ETH-vault deposit through the margin: the preview's shares and health factor match the sent transaction's result
- [ ] ETH-vault withdrawal through the margin: USDC received is at least `min_usdc_out`; the shares burned match the preview
- [ ] SOL vault: Deposit sends the user to a wallet deposit and then `user_deposit_collateral`; Withdraw through the margin succeeds with the wrapped-SOL account created first
- [ ] TSLA and META: deposits show `MarketAfterHours` outside the stock session
- [ ] Perena USD* and Junior: price and "updated" time shown; a NAV older than one hour shows a warning; a Junior exit shows the 1% fee in "You receive"
- [ ] Errors: a deposit that breaks 1.10 shows the health-factor message; an old route shows "price moved"; an old fork shows the stale-price message
- [ ] The share-price job stores a point per vault per hour (five basis vaults, two Perena); APY windows show the fallback until they have enough history
- [ ] Lookup tables exist (one per basis vault, one for Vanna); every transaction stays under 1,232 bytes and 64 accounts
- [ ] "Preview · sample data" comes off a vault's page only once its share mint is registered on Vanna

On a fork, Phoenix's mark stays where it was when the fork started while Jupiter quotes today's pools: an old fork fails deposits with `SwapPriceTooLow`. Start a fresh one. The program side of these flows is covered by Vanna's tests: `cd tests/basis && cargo test` (19 tests) and `cargo test --test perena`.